

부록. 시스템 아키텍처

1. 개요

본 연구에서 개발한 자기공감 대화 애플리케이션은 사용자, 프론트엔드, 백엔드, 인공지능, 데이터 관리의 다섯 요소로 구성된다. 사용자는 웹 기반 프론트엔드를 통해서만 시스템과 상호작용하며, 프론트엔드는 백엔드와 HTTP API로 통신한다. 인공지능 모델과 데이터베이스는 백엔드를 매개로 해서만 접근되며, 프론트엔드가 이들에 직접 접근하는 경로는 존재하지 않는다.

이러한 계층 구조는 두 가지 목적을 가진다. 첫째, 위기 상황 감지, 대화 흐름 결정, 데이터 보관 여부 판단 등 안전 및 개인정보와 관련된 결정을 백엔드의 규칙 기반 코드에서 일관되게 수행함으로써, 생성형 AI의 출력에 의존하지 않도록 한다. 둘째, 사용자 데이터에 대한 접근을 단일 경로로 제한하여 인증과 권한 검사를 일관되게 적용할 수 있도록 한다.

2. 구성 요소별 역할과 구현

2.1 프론트엔드

프론트엔드는 사용자가 직접 이용하는 웹 화면으로, 언어 선택, 대화 시작, 성찰일지 작성, 저장 기록 조회 기능을 제공한다. 인터페이스는 중국어와 한국어를 지원하며, 사용자는 이용 중 언제든지 언어를 전환할 수 있다. 진행 중인 대화에서 언어를 전환하면 이후의 응답이 선택한 언어로 생성된다.

[표 A-1] 프론트엔드 구현 기술

구분	기술	용도
프레임워크	Vue 3, TypeScript	화면 및 컴포넌트 구성
빌드	Vite	개발 환경 및 배포용 번들 생성
라우팅 · 상태 관리	Vue Router, Pinia	화면 전환, 로그인 및 대화 상태 관리
다국어	vue-i18n	중국어 · 한국어 인터페이스
배포	Cloudflare Workers	정적 자산 배포

2.2 백엔드

백엔드는 프론트엔드의 요청을 처리하고 인공지능 및 데이터베이스와의 연동을 담당한다. 주요 기능은 사용자 및 세션 관리, 대화 목적 · 상태 및 활동 경로 관리, 맥락 조회 및 프롬프트 구성, AI API 연동 및 응답 처리, 활동 · 기록 · 사용 로그 관리이다.

프롬프트는 공통 원칙, 대화 목적별 흐름 지침, 고정된 응답 형식 요구사항의 세 부분으로 조합된다. 이 중 공통 원칙과 흐름 지침은 연구진이 관리자 페이지에서 코드 수정 없이 조정할 수 있으며, 모든 수정 내용은 버전으로 보관되고 각 AI 응답에는 생성 당시의 프롬프트 버전과 모델명이 함께 기록된다.

[표 A-2] 백엔드 구현 기술

구분	기술	용도
언어 · 프레임워크	Python 3.12, FastAPI, Uvicorn	API 서버 및 비동기 요청 처리
데이터 접근	SQLAlchemy 2 (비동기), Pydantic	데이터베이스 연동, 입출력 형식 검증

구분	기술	용도
인증	JWT, 비밀번호 해시	아이디 · 비밀번호 기반 로그인
암호화	Fernet (cryptography)	저장 기록 및 성찰일지 암호화
배포	Docker	서버 환경 구성

2.3 인공지능

인공지능은 OpenAI API를 통해 연동되며, 응답은 사전에 정의된 JSON 형식(응답 문장과 선택지)으로 반환된다. 기본 모델은 gpt-5.6-terra이며, 관리자는 최근 출시된 모델 가운데 실제 호출이 확인된 모델로 변경할 수 있다.

인공지능이 담당하는 역할은 상황 · 맥락의 이해 및 요약, 감정 및 필요 탐색 질문 생성, 자기수용 표현 제안, 조절 활동 제안, 사용자의 수정 · 거부 내용 반영이다. 인공지능이 제시하는 감정, 해석, 표현은 모두 후보로 제시되며, 사용자가 확인한 내용만 대화 맥락으로 저장된다.

2.4 데이터 관리

데이터는 관계형 데이터베이스(SQLite)에 저장된다. 저장 대상은 사용자 동의 정보, 대화 기록, 성찰일지, 사용 일시 및 횟수 기록이다. 사용자가 저장을 선택한 기록과 성찰일지는 암호화되어 보관되며, 대화 기록은 사용자의 보관 동의 여부에 따라 대화 종료 시 삭제된다.

3. 대화 처리 절차

한 차례의 대화는 다음 절차에 따라 처리된다.

1. 사용자가 프론트엔드에서 메시지를 입력하면 백엔드로 전달된다.
2. 백엔드는 먼저 규칙 기반의 위기 키워드 검사를 수행한다. 위기 신호가 감지되면 인공지능을 호출하지 않고 사전에 검증된 지원 기관 정보를 즉시 제공한다.
3. 위기 신호가 없는 경우, 사용자가 선택한 대화 목적과 자기비판 수준에 따라 대화 흐름을 결정한다.
4. 데이터베이스에서 최근 대화 내용과 사용자가 확인한 맥락을 조회하여 프롬프트를 구성하고, 인공지능에 응답을 요청한다.
5. 인공지능이 생성한 응답과 선택지를 저장한 후 프론트엔드에 전달하며, 프론트엔드는 이를 대화 말풍선과 선택 카드로 표시한다.

4. 설계 원칙

[표 A-3] 시스템 설계 원칙

원칙	구현 방식
안전 판단의 코드화	위기 감지와 지원 정보 제공을 인공지능이 아닌 규칙 기반 코드로 수행한다.
사용자 결정권 보장	인공지능의 해석은 후보로만 제시하며, 사용자가 확인 · 수정 · 거부할 수 있다.
동의 기반 데이터 보관	기록은 사용자가 저장을 선택한 경우에만 보관하며, 저장 시 암호화한다.
계정 기반 접근 통제	모든 이용은 로그인을 전제로 하며, 사용자는 본인의 데이터에만 접근할 수 있다.
연구진에 의한 조정 가능성	프롬프트와 사용 모델을 관리자 페이지에서 조정할 수 있으며, 변경 이력과 응답별 사용 버전을 기록한다.

5. 핵심 구현 코드

본 절은 앞서 기술한 설계 원칙이 실제로 구현된 부분을 소스 코드에서 발췌한 것이다. 각 코드는 실제 시스템의 코드를 그대로 옮겼으며, 파일 경로는 저장소의 **backend/** 기준이다.

5.1 위기 신호 감지

위기 신호는 인공지능의 판단에 의존하지 않고, 중국어와 한국어의 위기 표현 목록을 정규식으로 대조하여 판정한다. 결과는 위기(CRISIS), 주의(WATCH), 해당 없음(NONE)의 세 단계로 구분된다.

[코드 A-1] 위기 표현 목록 및 판정 함수 (app/services/safety.py)

```
_CRISIS_PATTERNS_ZH = [
    r"不想活", r"自杀", r"想死", r"结束生命", r"伤害自己", r"割腕", r"想杀", r"杀了他", r"杀了她",
]
_CRISIS_PATTERNS_KO = [
    r"죽고\s*싶", r"자살", r"자해", r"삶을\s*끝", r"죽이고\s*싶", r"살고\s*싶지\s*않",
]

def screen_text(text: str) -> RiskLevel:
    if _matches(text, _CRISIS_PATTERNS_ZH) or _matches(text, _CRISIS_PATTERNS_KO):
        return RiskLevel.CRISIS
    if _matches(text, _WATCH_PATTERNS_ZH) or _matches(text, _WATCH_PATTERNS_KO):
        return RiskLevel.WATCH
    return RiskLevel.NONE
```

이 검사는 매 대화 턴에서 인공지능을 호출하기 전에 수행되며, 위기로 판정되면 인공지능을 거치지 않고 검증된 지원 기관 정보를 반환한다.

[코드 A-2] 대화 처리 시 위기 검사 순서 (app/services/dialogue_state.py)

```
risk = RiskLevel.NONE
if user_text:
    risk = safety.screen_text(user_text)
    session.self_criticism_level = safety.bump_self_criticism(user_text, session.self_criticism_level)
    if risk == RiskLevel.CRISIS:
        return await _crisis_turn(db, language)
```

5.2 대화 흐름 결정

사용자가 선택한 대화 목적에 따라 대화 흐름을 결정하되, 자기비판 수준이 기준값(0.5) 이상이면 「안정화」를 선택한 경우를 제외하고 자기친절 흐름을 우선 적용한다.

[코드 A-3] 대화 목적과 흐름의 대응 및 흐름 결정 함수 (app/prompts/router.py)

```

_FLOW_MAP = {
    DialogueIntent.VENT: registry.FLOW_EMOTION_EXPLORATION,
    DialogueIntent.ORGANIZE: registry.FLOW_EMOTION_EXPLORATION,
    DialogueIntent.STABILIZE: registry.FLOW_STABILIZATION,
    DialogueIntent.METHOD: registry.FLOW_RECOVERY_PLAN,
}

def flow_key_for(intent: DialogueIntent, self_criticism_level: float) -> str:
    if self_criticism_level >= 0.5 and intent != DialogueIntent.STABILIZE:
        return registry.FLOW_SELF_KINDNESS
    return _FLOW_MAP.get(intent, registry.FLOW_EMOTION_EXPLORATION)

```

5.3 프롬프트 구성

시스템 프롬프트는 공통 원칙(13개 항목), 대화 흐름별 지침, 응답 형식 요구사항, 응답 언어 지정의 순서로 조합된다. 공통 원칙과 흐름 지침은 관리자 페이지에서 수정할 수 있으나, 응답 형식과 응답 언어 지정은 코드에 고정되어 있어 수정되지 않는다.

[코드 A-4] 공통 원칙 조합 및 시스템 프롬프트 구성 (app/prompts/principles.py, app/prompts/base.py)

```

def build_rules_text(bodies: dict[str, str], language: Language) -> str:
    parts = [f" [{TITLES[key][language]}] \n{bodies[key].strip()}" for key
in KEYS if bodies.get(key, "").strip()]
    return "\n\n".join(parts)

def build_system_prompt(rules_text: str, flow_instructions: str, language: Language, flow_format: str) -> str:
    return (
        f"{rules_text}\n\n---\n{flow_instructions}\n\n---\n{FORMAT_RULES[language]}\n{flow_format}"
        f"\n{REPLY_LANGUAGE[language]}"
    )

```

인공지능에는 다음의 응답 형식이 함께 전달되며, 응답은 사용자에게 보여 줄 문장(reply_text)과 선택지(candidates)로 구성된 JSON으로 받는다.

[코드 A-5] 응답 형식 지정 (app/services/llm.py)

```

RESPONSE_INSTRUCTIONS = (
    "\n\n---\n"
    "输出格式 / Output format: 只返回一个 JSON 对象 / return exactly one JSON object, "
    '{"reply_text": string, "candidates": [{"type": string, "items": [{"id": string, "label": string}]}, '

```

```

    "fields": object?}}}. reply_text 是给用户看的简短消息; candidates 是可选的
    结构化建议, '
    "没有则给空数组。不要在 JSON 之外输出任何文字。"
)

```

5.4 사용자 확인 내용의 반영

인공지능이 제시한 감정, 표현, 계획은 사용자가 선택하거나 수정한 경우에만 대화 맥락(`confirmed_context`)에 저장된다. 이 함수는 대화 맥락이 기록되는 유일한 경로이며, 확인되지 않은 인공지능의 출력은 저장되지 않는다.

[코드 A-6] 사용자 확인 내용의 반영 (`app/services/dialogue_state.py`, 일부)

```

def merge_confirmation(session: ConversationSession, confirmation:
dict[str, Any]) -> None:
    ctx = dict(session.confirmed_context or {})
    card_type = confirmation.get("card_type")
    selected = confirmation.get("selected_labels", [])
    custom = confirmation.get("custom_text")
    values = [*selected, *( [custom] if custom else [] )]

    if card_type == "intent_options" and selected:
        ctx["last_intent"] = selected[0]
    elif card_type == "emotion_options":
        ctx["emotions"] = list({*ctx.get("emotions", []), *values})
    elif card_type == "self_kindness_options" and values:
        ctx.setdefault("self_kindness_texts", []).append(values[0])
    # (안정화 활동, 회복 행동, 행동 계획, 상황-감정-행동 요약도 같은 방식으로 반영)

    session.confirmed_context = ctx

```

5.5 동의 기반 데이터 보관과 접근 통제

대화 종료 시 사용자가 대화 기록 보관에 동의하지 않았다면 대화 내용을 삭제한다. 또한 모든 대화 관련 요청은 로그인한 사용자 본인의 대화인지 확인한 후에만 처리되며, 다른 사용자의 대화는 존재하지 않는 것으로 응답하여 대화 식별자를 추측하는 시도를 막는다.

[코드 A-7] 대화 종료 시 동의 여부에 따른 삭제 (`app/api/consent.py`, 일부)

```

consent = session.consent or {}
if not consent.get(ConsentCategory.DIALOGUE_HISTORY.value, False):
    await db.execute(delete(Message).where(Message.session_id ==
session.id))

session.ended_at = datetime.now(timezone.utc)

```

[코드 A-8] 본인 대화 확인 (`app/api/deps.py`)

```
async def get_owned_session(
    session_id: str,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(get_current_user_required),
) -> ConversationSession:
    session = await get_session_or_404(session_id, db)
    if session.user_id != user.id:
        raise HTTPException(status_code=404, detail="session not found")
    return session
```